



UNIVERSAL HEALTH RECORDS DATABASE

PROJECT SUMMARY

CIS-552 DATABASE DESIGN

GROUP 6

PREPARED BY

Steven Quintana (Roll Number: 54)

Jeevan Kumar (Roll Number: 6)

Sai Bhaskar (Roll Number: 33)



CONTENTS

1. Problem Statement.....	3
2. Use Cases.....	3
3. Technical Obstacles.....	3
4. Tools and Technology.....	4
5. Modelling the Data.....	6
6. Initialising the MongoDB Database.....	7
7. Seeding Initial Patient Data.....	8
8. Further Data Cleaning in Compass.....	10
9. Frontend Creation.....	13
10. Server Creation.....	15



INTRODUCTION

The Universal Health Data System is a document based NoSQL database which maintains patient health records in one location from many different providers. Using a document based NoSQL database, implemented with MongoDB, best achieves the goal of storing large amounts of health data, where the structure must be flexible due to the individualised nature of patients' conditions, history, and lifestyles.

The system helps to streamline historical health data collection of patients coming from a different provider, and give practitioners robust details on a patient in order to better analyse and diagnose any problems at hand.

The system also offers end points to approved research labs in order to have a centralised location to analyse large health data sets and provide more robust and accurate results for health issues facing the world today.

1. Problem Statement

The exchange and centralization of health data remains a critical step yet to be taken in our healthcare system. Due to regulations, patient data is still dispersed amongst different carriers in the health system, making it difficult for providers to offer robust health recommendations and catch early signs of health issues. Some government and private organisations are currently trying to solve this issue, such as the company CommonWell. CommonWell relies heavily on ETLs in order to provide access to a central database from multiple different interfaces.

Because of the dynamic nature of notes within offices and patient symptoms/lifestyles, a universal interface linked to a document based NoSQL database can provide more flexibility and maintainability. This also helps to achieve the sub-goal of creating access points for labs to query large amounts of data for research purposes.

2. Use Cases

- Doctor to Patient Care: The database can be accessed easily with the GUI implemented by the doctor to pull down new client health history and sync new data in real time with new data from the visit.
- Research: The data within the database can also be accessed via end points to pre-approved research groups for purposes of finding cures to health issues.

3. Technical Obstacles

1. Development of a database that can take in multiple data structures and load them into one database requires the development of robust and detailed transformation scripts that can transform data in an automated way.
2. HIPPA laws require the digital sharing of information to be authorised by every patient and great detail must be taken into account when designing a database of digital health records.

4. Tools and Technology

MongoDB

MongoDB was chosen because it is a document based NoSQL database management system, which is best suited for the business use case. MongoDB offers a user-friendly interface to create and modify data models locally via MongoDB Compass, which can then be deployed to a remote server.

Python

Python is used as the programming language for the applications backend and will hold the logic which consolidates user input and queries the Mongo database based on user actions.

PyMongo

The python library which will allow the backend to perform CRUD operations and more queries on the Mongo database.

HTML/CSS/Javascript

For the frontend we chose web based technologies in order to simulate creating a web app server that would need to connect and interact with a frontend app via HTTPS queries.



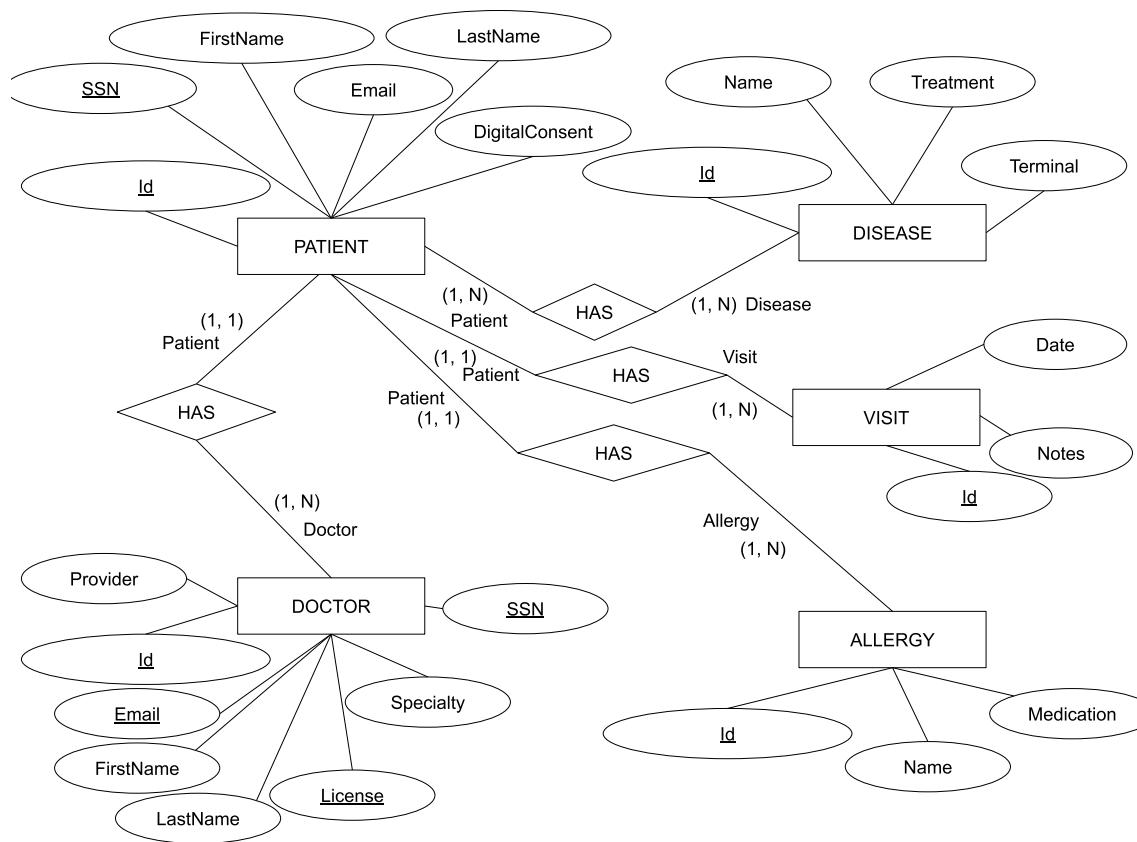
IMPLEMENTATION

The implementation of the Healthcare Records database was done over a timespan of three weeks by three developers. The project approach was to use a high level modelling approach, which then used various technologies to implement that model as the implementation became more granular. The implementation structure detailed below is a representation of this approach.

- 1. Data Model Design**
- 2. Data Collection and Transformation**
- 3. Frontend and Server Creation**

1. Modelling the Data

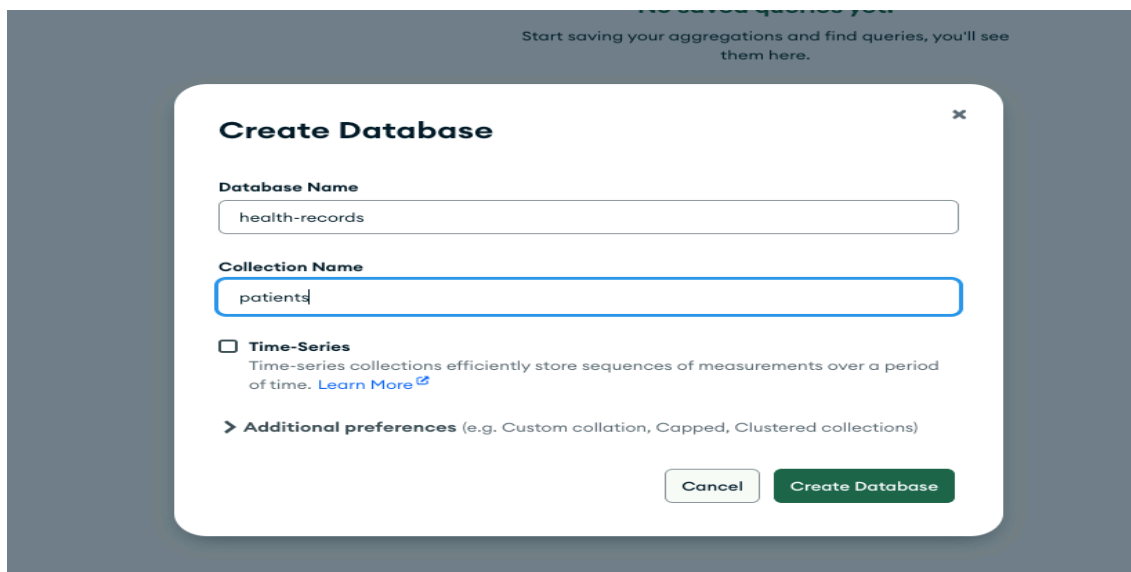
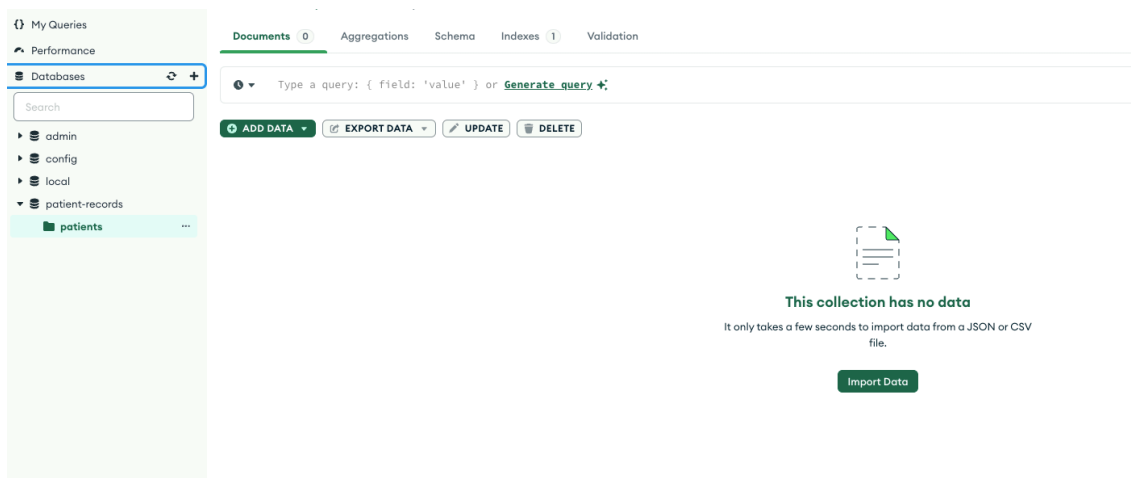
To begin, the data was modelled in a relational diagram. Though the data structures would ultimately be document based, the model is a helpful visual to understand the connections between the different collections. The biggest difference in the implementation to a document based type structure was to create a list of related `_id`'s within a collection to create the relationships. For instance, the Doctor collection holds a list of Patients `_ids`.



2. Initialising the MongoDB Database

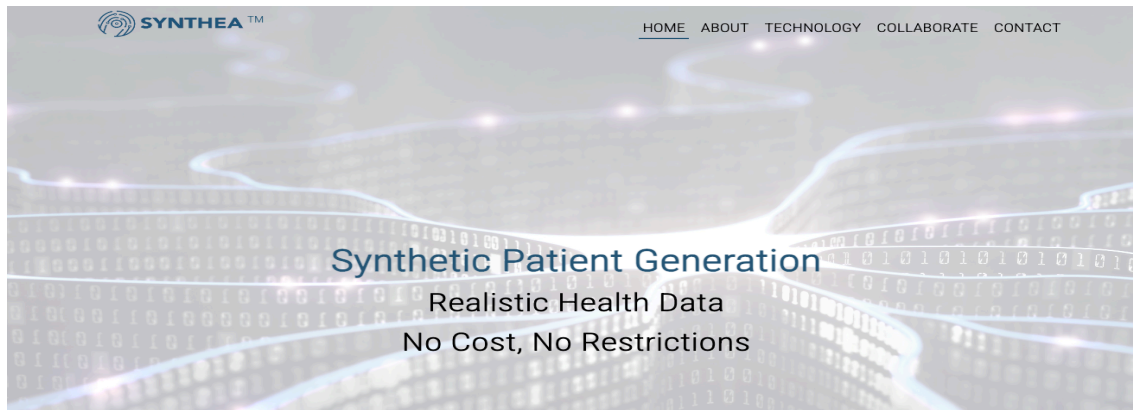
To begin, the group started by selecting where we wanted the data to be stored. Because the purposes of this project were scaled down, choosing to work in MongoDB Compass was the best option, however at scale a cloud provider who services document databases like AWS integrating with MongoDB integration would be needed and data would need to be moved from on premise to off.

After choosing Compass, a new document database was initialised as 'patient-records' along with the first collection called 'patients', which holds the primary data of this database.



3. Seeding Initial Patient Data

To seed the initial Synthea was chosen as an open source data set of synthetic health record patient data. This data was a vast amount of patient and health data in multiple related tables. A transformation script was created in order to specifically the patient data, transform/limit the data to match our data structures in MongoDB where dummy allergy and disease data was added to replace the current data. In the future, transformation scripts would be included to apply all data to the database.



CSV FILES			
▼ CSV	Folder	Today at 9:08 AM	
observations.csv	43.1 MB CSV Document	Today at 9:08 AM	
patients.csv	333 KB CSV Document	Today at 9:08 AM	
immunizations.csv	2.2 MB CSV Document	Today at 9:08 AM	
devices.csv	17 KB CSV Document	Today at 9:08 AM	
encounters.csv	16.5 MB CSV Document	Today at 9:08 AM	
careplans.csv	673 KB CSV Document	Today at 9:08 AM	
conditions.csv	1.1 MB CSV Document	Today at 9:08 AM	
organizations.csv	170 KB CSV Document	Today at 9:08 AM	
procedures.csv	5.7 MB CSV Document	Today at 9:08 AM	
allergies.csv	70 KB CSV Document	Today at 9:08 AM	
payers.csv	2 KB CSV Document	Today at 9:08 AM	
supplies.csv	49 bytes CSV Document	Today at 9:08 AM	
imaging_studies.csv	205 KB CSV Document	Today at 9:08 AM	
payer_transitions.csv	341 KB CSV Document	Today at 9:08 AM	
providers.csv	1 MB CSV Document	Today at 9:08 AM	
medications.csv	10.9 MB CSV Document	Today at 9:08 AM	

```

# Path to input csv and output json files
input_csv_dir = 'output/csv'
output_json_dir = 'output/json'

# Function to convert CSV to JSON with selected fields and additional fields
def convert_csv_to_json(csv_file_path, limit=None, fields=None):
    df = pd.read_csv(csv_file_path)
    df = df.fillna("") # Replace NaN values with empty string

    # Rename columns to all lower case
    df.columns = [col.lower() for col in df.columns]

    # Select only the specified fields (excluding the new fields to be added)
    if fields:
        df = df[[field for field in fields if field in df.columns]]

    # Convert DataFrame to list of dictionaries
    json_data = df.to_dict(orient='records')

    # Apply limit if provided
    if limit is not None:
        json_data = json_data[:limit]

    return json_data

# Path to the specific CSV file
patients_csv_file = 'patients.csv'
file_path = os.path.join(input_csv_dir, patients_csv_file)

# Specify the fields to include, including new fields
fields = [
    '_id', 'birthdate', 'ssn', 'prefix', 'firstName', 'lastName', 'gender',
    'address', 'city', 'state', 'county', 'zip'
]

# Convert CSV to JSON with a limit of 50 objects and specified fields
json_data = convert_csv_to_json(file_path, limit=50, fields=fields)

# List of diseases and allergies for random data generation
diseases = [
    'Hypertension', 'Diabetes', 'Asthma', 'Cancer', 'Heart Disease', 'Chronic Obstructive Pulmonary Disease (COPD)',
    'Arthritis', 'Stroke', 'Kidney Disease', 'Alzheimer's Disease', 'Obesity', 'Liver Disease'
]

allergies = [
    'Penicillin', 'Peanuts', 'Shellfish', 'Pollen', 'Dust Mites', 'Insect Stings', 'Latex', 'Mold', 'Pet Dander',
    'Food Additives', 'Eggs', 'Milk', 'Soy', 'Wheat'
]

# Add dummy data that was not available in dataset, but need for database
def add_dummy_data(records, diseases, allergies):
    for record in records:
        record['diseases'] = random.sample(diseases, 2) # Select 2 random diseases
        record['allergies'] = random.sample(allergies, 2) # Select 2 random allergies
        # Generate email using the first name
        first_name = record['first_name'].lower()
        record['email'] = f"{first_name}@example.com"
        record['digitalConsent'] = True # Assuming consent is given

    return records

```

Databases

Search

- admin
- config
- local
- patient-records
 - patients

Type a query: (field: 'value') or Generate query

ADD DATA EXPORT DATA UPDATE DELETE

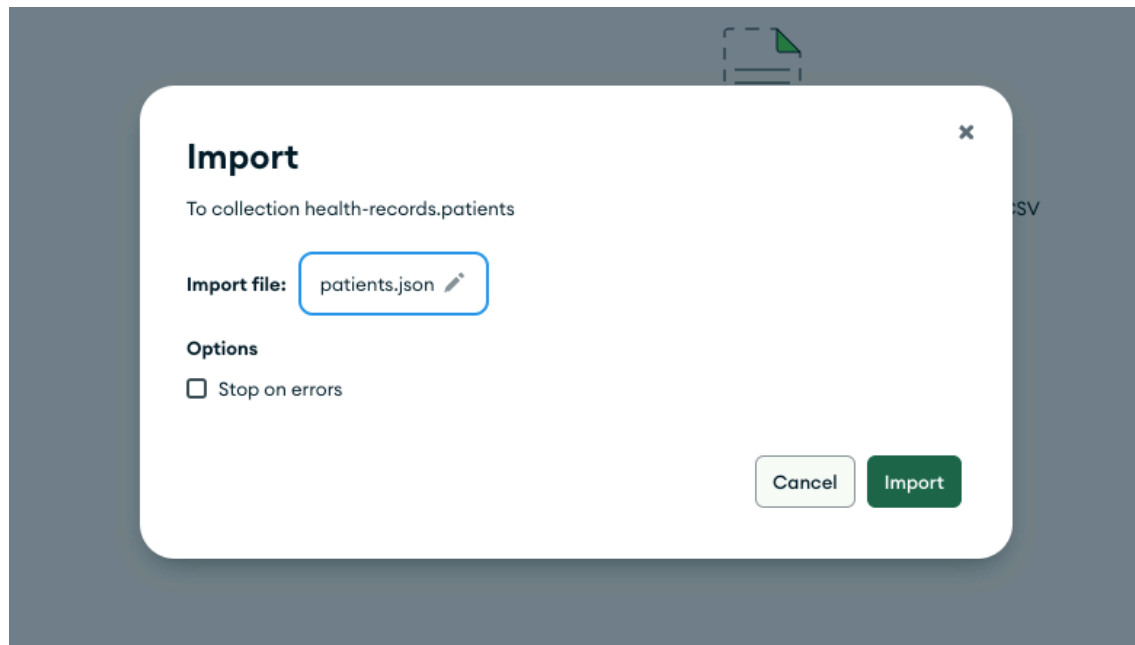
1 - 20 of 171

14: "10084d9-9a81-4bad-86c2-da375d99b469"
 BIRTHDATE: "1989-05-15"
 DEATHDATE: ""
 SSN: "999-79-6808"
 DRIVERS: "999984236"
 PASSPORT: "X193772608"
 PREFIX: "Mr."
 FIRST: "Jose Eduardo181"
 LAST: "Gomez296"
 SUFFIX: ""
 MAIDEN: ""
 MARITAL: "M"
 RACE: "White"
 ETHNICITY: "Hispanic"
 GENDER: "M"
 BIRTHPLACE: "Marigot, Saint Andrew Parish, DM"
 ADDRESS: "421 Ballistreri Way Unit 19"
 CITY: "Chicago"
 STATE: "Massachusetts"
 COUNTY: "Wampden County"
 ZIP: 0613
 LAT: 42.22835382310942
 LON: -72.9629505068082
 HEALTHCARE_EXPENSES: 271227.08
 HEALTHCARE_COVERAGE: 1334.68
 medications: Array (2)
 paper_transitions: Array (1)
 imaging_studies: Array (empty)
 allergies: Array (empty)
 procedures: Array (1)
 conditions: Array (1)
 careplans: Array (empty)
 encounters: Array (6)
 devices: Array (empty)
 immunizations: Array (6)
 observations: Array (79)
 Hide 11 fields

14: "934a3e3b-2def-4559-bb2a-7858888aee09"
 BIRTHDATE: "1993-11-14"
 DEATHDATE: ""
 SSN: "999-73-5361"
 DRIVERS: "999962402"
 PASSPORT: "X198875464X"

Import completed.
171 documents imported.

Once the data was transformed from the csv file into a json file for the Patients collection, the file was used to do a mass import into the local MongoDB database instance to see the initial data of all the patients.



4. Further Data Cleaning in Compass

Once the data was imported into the database, it was decided to change all field descriptions to lower case instead of upper case, and to reduce the amount of fields. To do this an Aggregate Function was created in Compass in order to create a pipeline that transformed all fields from the Compass UI, however an ETL script is preferred for automation, and the prior script was updated to match these changes.

The screenshot displays the MongoDB Compass interface for a database named 'health-records' on 'localhost:27017'. The 'patients' collection is selected, showing three documents. Each document contains fields like '_id', '_id_', 'birthdate', 'ssn', 'first', 'last', 'prefix', 'gender', 'address', 'city', 'state', 'county', 'zip', 'diseases', 'allergies', 'email', and 'digitalConsent'.

Below the documents, an aggregation pipeline is shown in the 'Aggregations' tab. The pipeline consists of three stages: '\$addFields', '\$project', and '\$lookup'. The '\$addFields' stage adds new fields based on existing ones. The '\$project' stage restructures the output. The '\$lookup' stage performs a join operation.

```

1. {
2.   "$addFields": {
3.     "birthdate": "$BIRTHDATE",
4.     "ssn": "$SSN",
5.     "first": "$FIRST",
6.     "last": "$LAST",
7.     "diseases": "$DISEASES",
8.     "allergies": "$ALLERGIES",
9.     "email": "$EMAIL",
10.    "digitalConsent": "$DIGITAL_CONSENT"
11.  },
12. },
13. },
14. {
15.   "$project": {
16.     "_id": 0,
17.     "birthdate": 0,
18.     "ssn": 0,
19.     "first": 0,
20.     "last": 0,
21.     "prefix": 0,
22.     "gender": 0,
23.     "address": 0,
24.     "city": 0,
25.     "state": 0,
26.     "county": 0,
27.     "zip": 0,
28.     "diseases": 0,
29.     "allergies": 0,
30.     "email": 0,
31.     "digitalConsent": 0
32.   },
33. },
34. }

```

The 'PIPELINE OUTPUT' section shows the result of the aggregation, displaying three documents with the transformed fields. The output documents are identical to the input documents, indicating that the aggregation pipeline is currently a no-op or the fields are being mapped directly.

```
31 "DIGITAL CONSENT": {
32   },
33 },
34 {
35   // Fold code block
36   //
37   * into: The target collection.
38   * on: Fields to identify.
39   * let: Defined variables.
40   * whenMatched: Action for matching docs.
41   * whenNotMatched: Action for non-matching docs.
42   //
43   {
44     into: "patients",
45     on: "_id",
46     whenMatched: "replace",
47     whenNotMatched: "insert"
48   }
49 }
50 ]
```

```
{
  "_id": ObjectId("66b2b1de6e93f9ca95c2d3e1"),
  "birthdate": "1992-06-02",
  "ssn": "999-27-3385",
  "firstName": "Jayson88",
  "lastName": "Fate1536",
  "diseases": Array (2),
  "allergies": Array (2),
  "email": "jayson88@example.com",
  "digitalConsent": true
}
```

```
{
  "_id": ObjectId("66b2b1de6e93f9ca95c2d3e2"),
  "birthdate": "1978-05-27",
  "ssn": "999-85-4926",
  "firstName": "Mariana775"
}
```

The \$merge operator will cause the pipeline to persist the results to the specified location.

ES",

DIGITAL CONSENT"

A write operation will occur

This pipeline will execute a [\\$merge](#) operation, altering "health-records.patients".

Do you wish to proceed?

Cancel Yes, run pipeline

```
birthdate: "1989-05-25"
ssn: "999-76-6866"
firstName: "José Eduardo181"
lastName: "Gómez206"
diseases: Array (2)
  (2)
do181@example.com"
true

2b1de6e93f9ca95c2d
1-14"

1"
"
)
(2)
example.com"
digitalConsent: true
```

5. Frontend Creation

For the frontend, HTML/CSS/Javascript were the technologies used. These technologies were chosen because the project is meant to be a server that can consume data from any provider and load it into the universal database. A web application was best to display this functionality because it could be connected via the Fetch API using HTTPS requests.

```

</style>
</head>
<body>
<div class="container" id="loginScreen">
  <h2>Doctor Login</h2>
  <form>
    <label for="loginUsername">Username (Email):</label>
    <input type="text" id="loginUsername" required>
    <label for="loginPassword">Password:</label>
    <input type="password" id="loginPassword" required>
    <button type="button" onclick="login()">Login</button>
    <button type="button" onclick="showSetupScreen()">Create Account</button>
  </form>
</div>

<div class="container hidden" id="setupScreen">
  <h2>Setup Doctor Account</h2>
  <form>
    <label for="doctorFirstName">First Name:</label>
    <input type="text" id="doctorFirstName" required>
    <label for="doctorLastName">Last Name:</label>
    <input type="text" id="doctorLastName" required>
    <label for="providerName">Provider Name:</label>
    <input type="text" id="providerName" required>
    <label for="doctorSSN">SSN:</label>
    <input type="text" id="doctorSSN" required>
    <label for="licenseNumber">License Number:</label>
    <input type="text" id="licenseNumber" required>
    <label for="specialty">Specialty:</label>
    <input type="text" id="specialty" required>
    <label for="doctorEmail">Email:</label>
    <input type="email" id="doctorEmail" required>
    <label for="doctorPassword">Password:</label>
    <input type="password" id="doctorPassword" required>
    <button type="button" onclick="createAccount()">Create Account</button>
    <button type="button" onclick="showLoginScreen()">Back to Login</button>
  </form>
</div>

<div class="container hidden" id="doctorView">
  <h2>Doctor View</h2>
  <div class="actions">
    <button onclick="showCreatePatientScreen()">Create Patient</button>
    <button onclick="showLookupPatientsScreen()">Lookup Patient</button>
    <button onclick="showLoginScreen()">Back to Login</button>
    <button onclick="deleteAllData()">Delete All Patients</button>
  </div>
</div>
</div>

```

```

<script>
// Function to send POST request for doctor login
function login() {
  const email = document.getElementById('loginUsername').value;
  const password = document.getElementById('loginPassword').value;

  fetch('/login', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json'
    },
    body: JSON.stringify({ email, password })
  })
  .then(response => response.json())
  .then(data => {
    if (data.success) {
      localStorage.setItem('currentUser', data.user);
      showDoctorView();
    } else {
      alert('Login failed. Please check your credentials.');
```

Doctor Login

Username (Email):

Password:

Login

Create Account

Lookup Patient

Patient List

Patient 1

First Name:

Steven

Last Name:

Quintana

SSN:

444-333-2222

Diseases:

Allergies:

Peanuts

Email:

steve@umass.edu

Digital Consent:

true

Update

Delete

Back to Doctor View

Doctor View

Create Patient

Lookup Patient

Back to Login

Delete All Patients

6. Server Creation

For the frontend, HTML/CSS/Javascript were the technologies used. These technologies were chosen because the project is meant to be a server that can consume data from any provider and load it into the universal database. A web application was best to display this functionality because it could be connected via the Fetch API using HTTPS requests.

Flask

The Flask Python library was used in order to consume and return data from the frontend via HTTPS requests. This technology is open-source with many contributors, is a relatively mature library having been around for over 10 years, and is popular among python developers, making it easy to use and stable for the purposes of this project. The following CRUD operations on the python server were implemented with the technology.

- **Doctor Routes:**
 - *Get One Doctor*
 - *Create One Doctor*
- **Patient Routes:**
 - *Get Many Patients*
 - *Update One Patient*
 - *Delete One Patient*
 - *Delete Many Patients*

```
# Doctor login
@app.route('/login', methods=['POST'])
def login():
    data = request.json
    email = data.get('email')
    password = data.get('password')
```

```
# Create a doctor
@app.route('/doctors', methods=['POST'])
def create_doctor():
    new_doctor = request.json
```

```
# Create a patient
@app.route('/patients/<doctor_id>', methods=['POST'])
def create_patient(doctor_id):
    # Extract patient data from the request
    new_patient = request.json
```

```
# Get all patients for a doctor
@app.route('/patients/<doctor_id>', methods=['GET'])
def get_patients(doctor_id):
    doctor = doctors_collection.find_one({'_id': ObjectId(doctor_id)})
```



```

        return jsonify({'error': 'Patient record not found'}), 404

# Update one patient record
@app.route('/patients/<patient_id>', methods=['PUT'])
def update_patient(patient_id):
    updated_patient = request.json

```

```

        return jsonify({'message': 'Patient record not found'}), 404

# Delete one patient record
@app.route('/patients/<patient_id>', methods=['DELETE'])
def delete_patient(patient_id):
    # Convert patient_id to ObjectId
    patient_obj_id = ObjectId(patient_id)

```

PyMongo

The PyMongo library was used to create queries that would fetch data from the MongoDB database created locally. To accomplish this a server file was created to hold all server logic which would connect frontend requests to the database. In this file a connection to the local database instance was created using MongoClient, a class within the PyMongo library used to create connection strings to MongoDB instances.

```

# Connect to MongoDB
client = MongoClient('mongodb://localhost:27017/')
db = client['health-records']
patients_collection = db['patients']
doctors_collection = db['doctors']

```

Once the connection to the local server is made, the collection variables (which are reference pointers to the collections within the database) are used to execute data queries.

A simple example of a MongoDB query that was created for the Health Records Database was finding a doctor using their Email and Password. Below is an example of this query executed using the PyMongo library.

```

doctor = doctors_collection.find_one({'email': email, 'password': password})

if doctor:
    return jsonify({'success': True, 'message': 'Login successful', 'user': dumps(doctor)}), 200
else:
    return jsonify({'success': False, 'message': 'Invalid login credentials'}), 404

```

The doctor information is fetched using the find_one() function, since we are only looking for one doctor, and the result is stored into a doctor variable as an object. That object can then be used to return the data using Flask if the data does exist.

Complexity in PyMongo Queries

This database overall had fairly simple query requests. However, MongoDB and PyMongo together handle complex queries well in a document structure.

For instance, relationships in a document based database can be handled using lists. Updating these lists from a code perspective is straightforward, using an **update_one()** function and passing the document id you wish to update along with a **\$push** or **\$pull** statement to add or remove an item from the specific list, all in an object format.

```
update_result = doctors_collection.update_one(
    {'_id': ObjectId(doctor_id)},
    {'$push': {'patients': new_patient_id}} # Use $push to add the patient ID to the array
)
```

```
# Remove the patient ID from the doctor's patient list
doctors_collection.update_one(
    {'patients': patient_id},
    {'$pull': {'patients': patient_id}}
)
```



CONCLUSION

This project successfully created and implemented a MongoDB Universal Patient Healthcare Records database with the basic CRUD operations needed in order to Create, Fetch, Update, and Delete the data of patients. The goal of transforming patient data from other data sources, loading it into the universal database, and be able to interact with that data via a frontend application were accomplished as summarised below:

1. Creation of a Document-Based database via MongoDB:

- a. Thoughtful data modelling was used to understand the structure of the database needed, and then transformed into the practical implementation of the database using MongoDB.
- b. The document based DB allowed for flexibility of data when planning to handle data from different data sources going into the future.
- c. MongoDB Compass allowed for fast and easy testing locally, and quick visibility into the data.

2. Development of a Transformation Script:

- a. The ETL written successfully transformed a set of relational based patient data into the format necessary to achieve the project goals within a document based set of data.
- b. The aim and outcome was to transform data from any type, like a csv file, into a json file that was easily loaded into MongoDB.

3. Development of a Frontend to Interact with Patient Data:

- a. The frontend created using HTML/CSS/Javascript accomplished the goal of having a simple and intuitive GUI that a doctor could interact with in order to find and update patient records.
- b. The web technology used is meant to mimic further instances of frontend applications that could interact with the server from many different providers.

4. Development of a Server to Connect the Frontend to the Database:

- a. The server successfully implemented the crucial task of being able to run an application that could be used by multiple frontend end applications to get patient information and update it.
- b. Flask and PyMongo in combination provided an efficient and scalable codebase in order to implement the database, and make updates as it grows over time.



FUTURE CONSIDERATIONS

Due to the timeframe, this project is representative of a viable working project to meet necessary requirements of a functioning healthcare records database. In order to create a more robust database, future requirements to be implemented have been considered and detailed below.

1. Add further collections:

- a. The collections used represent what is needed for a doctor to pull down client data, however further relationships can be formed between Allergy, Disease, and Provider collections.
- b. These collections would provide a better way to form relationships in the doctor collection, using a list of Allergy(etc) ids instead of a list of strings.

2. Add Front End validations:

- a. Future enhancements will provide a more robust application user experience to increase validations on the frontend so invalid server calls are not made.
- b. Likewise, future implementations would include list dropdowns as opposed to comma separated input values.

3. Add further ETL scripts:

- a. As noted prior in the project summary, further focus will need to be around transformation scripts in order to universally load data into the database from multiple data sources.

4. Create a secure API for researchers to use:

- a. Finally creating security measures, clear error message handling, and robust documentation will be added so that the server API can be opened to select research groups.